Juniorprogrammierer.de

Go 3: Go routine

2024/25/26 – Sascha Stojanovic

Synchronous vs asynchronous

Synchornous

- tasks run one after another. Each task must finish before the next one starts
- Like standing in line at a coffee shop — you wait for the person ahead to finish before ordering

Asynchronous

- tasks can start without waiting for previous ones to finish. Multiple things happen at overlapping times
- Like ordering coffee online while doing laundry and cooking dinner — you don't wait for one to finish before starting another.

Concurrency in go

- Concurrency is the mechanism that enables asynchronous execution
- The relationship
  - Synchronous/Asynchronous = what happens (the behavior from your perspective)
  - Concurrency = how it's implemented (the technique that makes it possible)
- In Go specifically: goroutines and channels are the concurrency tools that let you write asynchronous code. Without concurrency primitives, you're stuck with synchronous execution

Go routine in action

Without go (Synchronous)

- sayHello() // Blocks for 2 seconds
- Main waits. Sequential execution.

With go (Concurrent)

- go sayHello() // Returns immediately
- Main continues. Both run in parallel.

Key Point: The go keyword launches a lightweight thread that runs independently without blocking the caller.

```
func sayHello() {
    time.Sleep(2 * time.Second)
    fmt.Println("Hello!")
}

func main() {
    fmt.Println("=== WITHOUT go keyword (synchronous) ===")
    start := time.Now()

    sayHello()  // BLOCKS here for 2 seconds
    fmt.Println("Main continues...")

    fmt.Printf("Total time: %v\n\n", time.Since(start))

    fmt.Println("=== WITH go keyword (concurrent) ===")
    start = time.Now()

    go sayHello()  // Does NOT block, runs in background
    fmt.Println("Main continues immediately!")

    time.Sleep(3 * time.Second)  // Wait for goroutine to finish
    fmt.Printf("Total time: %v\n", time.Since(start))
}
```

Exercize

Create a program that simulates 3 workers processing tasks concurrently

Requirements:

1. Write a function worker(id int) that:
   - Prints "Worker [id] starting"
   - Sleeps for 2 seconds (simulates work)
   - Prints "Worker [id] finished"
2. In main():
   - Launch 3 workers as goroutines
   - Print "All workers started"
   - Wait long enough for all to finish
   - Print "All workers done”

Expected Output:

```
Worker 1 starting
Worker 2 starting
Worker 3 starting
All workers started
Worker 1 finished
Worker 2 finished
Worker 3 finished
All workers done
```

Hint: Start with:

```
func worker(id int) {
    // your code here
}

func main() {
    // launch 3 workers with go keyword
}
```